数据结构
面试复习资料

目录

一、 O(n)的大 O 是什么意思?什么是时间复杂度?.....3
二、 解释一下顺序存储与链式存储.....3
三、 判断链表是否有环.....3
四、 平衡二叉树、二叉排序树、完全二叉树、二叉搜索树的区别及如何构造.....3
五、 B 树是什么?在数据库中有什么应用?.....4
六、介绍一下字符串匹配算法.....4
六、 最小生成树和最短路径用什么算法来实现?.....4
七、 如何由遍历序列构造一颗二叉树?.....6
八、 哈希表的概念、构造方法、哈希有几种类型?.....8
九、 如何解决哈希冲突.....9
十、 栈和队列的区别和内存结构.....10
十一、 数据结构中的栈和计算机里的栈有什么不同.....12
十二、 如何区分循环队列是队空还是队满?.....12
十三、 红黑树.....12
十四、 八大排序.....13
十五、 哈夫曼树.....15
十六、 链表反转.....16
十七、 贪心，动态规划，分治.....16
十八、 如何用两个栈实现队列，如何用两个队列实现栈？.....17
十九、 循环引用怎么解决.....17
二十、 什么是拓扑排序.....17

一、O(n)的大 O 是什么意思?什么是时间复杂度?

大 O 符号，又称为渐进符号，是用于描述函数渐近行为的数学符号。更确切地说，它是用另一个通常更简单的函数来描述一个函数数量级的渐近上界。

时间复杂度，是一个用于度量一个算法的运算时间的一个描述，本质是一个函数，它描述的只是代码执行时间随数据规模增长的变化趋势

二、解释一下顺序存储与链式存储

1、**顺序表**就是把线性表中的所有元素按照某种逻辑顺序，依次存储到从指定位置开始的一块连续的存储空间。

优点：可快速的存取表中的任一位置元素；无需为表示表中元素之间的逻辑关系而增加额外的存储空间。

缺点：插入和删除操作需要移动大量元素；当线性表长度变化比较大时，难以确定存储空间的容量；容易造成存储空间碎片。

2、**链表**用一组任意的存储单元存储线性表的数据元素（这组存储单元可以是连续的，也可以是不连续的），包括数据域和指针域，数据域存数据，指针域指示其后继的信息。

优点：节省空间，不会出现空间碎片；插入和删除数据快速，只用更改指针域。

缺点：存取元素慢，不支持任意存取的，需要从头走到尾。

三、判断链表是否有环

1、快慢指针，慢的每次一步，快的每次两步，有环的话他们一定会在环内相遇。长度的要点是求出入口点，慢指针走了 s，快指针就是 2s。

2、除此之外还有穷举法，每到一个节点就遍历它前面的节点。

四、平衡二叉树、二叉排序树、完全二叉树、二叉搜索树的区别及如何构造

1、**完全二叉树:** 只有最下面的两层结点度小于 2，并且最下面一层的结点都集中在该层最左边的若干位置。

2、**满二叉树：**全部节点度为 2

3、**二叉排序树**（二叉查找树）：左子树上的值都小于根结点的值，右子树上的值都大于根结点得值，左右子树都是二叉排序树。

4、**平衡二叉树（ALV）：**是一颗二叉排序树，左子树和右子树的差值不超过

1，左右子树都为平衡二叉树。

五、B 树是什么?在数据库中有什么应用?

B 树是一种自平衡的树，能够保持数据有序。每进一个数字就会动态调整一次，节点中值 m-1，节点数 m。

B 树这种数据结构可以用来描述外部存储。

Mysql 用的是 B+树，B+树内节点不存储数据，所有 data 存储在叶节点导致查询时间复杂度固定为 log n。而 B-树查询时间复杂度不固定，与 key 在树中的位置有关，最好为 O(1)。B+树叶节点两两相连可大大增加区间访问性。

六、介绍一下字符串匹配算法

暴力匹配法，用双循环，当主串与模式串开始相同进行比较，若有不同则外循环往后移一位，重新开始比较。内循环是 i+j，j 从 0 开始。

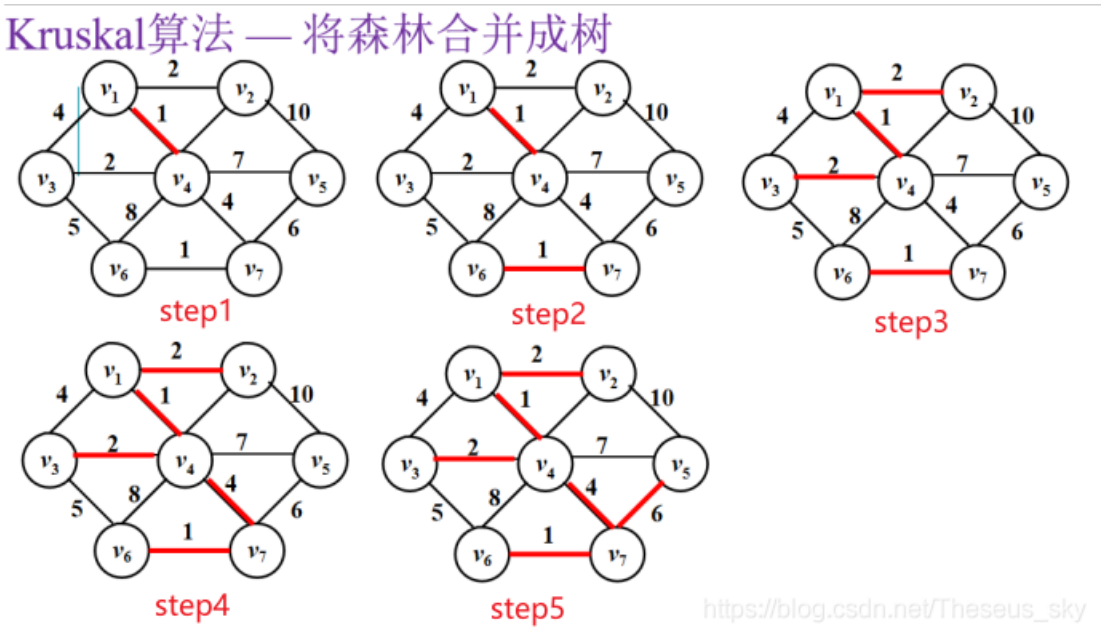
KMP：若每次都从头开始比较有些慢，适当的跳过重复部分可以加速，我们采用最长公共前后缀，next 数组。Next 先打造出来。

二分搜索：O（logn）有序数组。

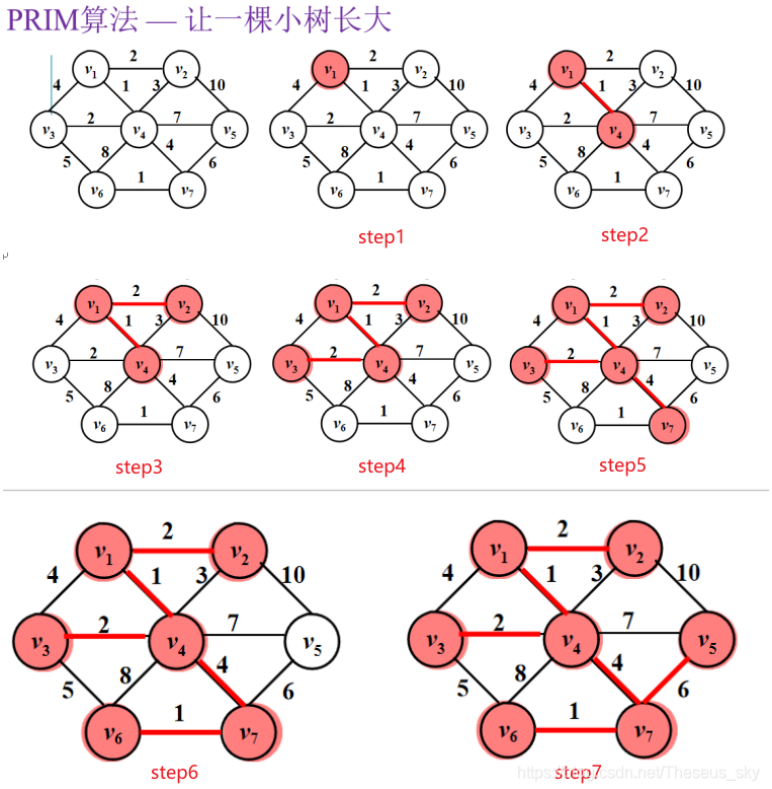
六、最小生成树和最短路径用什么算法来实现?

(一)最小生成树：（无向图）

1、Kruskal 算法：选边，从权重最小的边开始选，确保连接的节点是两个不同的树，不会连成环，直到有 n-1 个边被选择。



2、prim 算法：选点，之后从点集合中选择一个代价最小的边，也就是权值最小的点，直到拥有 n 个点或 n-1 个边。



1.Prim在稠密图中比Kruskal优，在稀疏图中比Kruskal劣。

2.Prim_heap在任何时候都有令人满意的的时间复杂度，但是代价是空间消耗极大。(以及代码很复杂>_

但值得说一下的是，时间复杂度并不能反映出一个算法的实际优劣。

竞赛题一般给的都是稀疏图，选择Prim_heap即可；如果觉得代码量太大，想要在Prim与Kruskal算法中选一个，那就选择Kruskal算法。

(二) **最短路径：**（有向图）

1、Dijkstra 算法：选临近点更新最短路径矩阵，求 s 到其他点的最短路径，与 s 临近的点的边权值放进矩阵，不相邻的记为∞远，在矩阵里找一个最近的点加入集合，他也有相邻点，更新矩阵，若出现更小的最短路径就更新，重复 step 直到全部点都计算出最短路径。时间复杂度 O（n^2）

2、FLoyd 算法：先给出一个大矩阵里面是每个点单独到其他点的距离，所以非相邻点都为∞，之后以每个节点为中心更新矩阵，更新自己能到达而其他节点到达不了的，需要注意这个节点要是到达不了更新的节点就不能去更新它。时间复杂度 O（n^3）

		A	B	C	D	E	F	G
以顶点A为中介点更新矩阵。	A	0	12	*	*	*	16	14
	B	12	0	10	*	*	7	26
	C	*	10	0	3	5	6	*
	D	*	*	3	0	4	*	*
	E	*	*	5	4	0	2	8
	F	16	7	6	*	2	0	9
	G	14	26	*	*	8	9	0

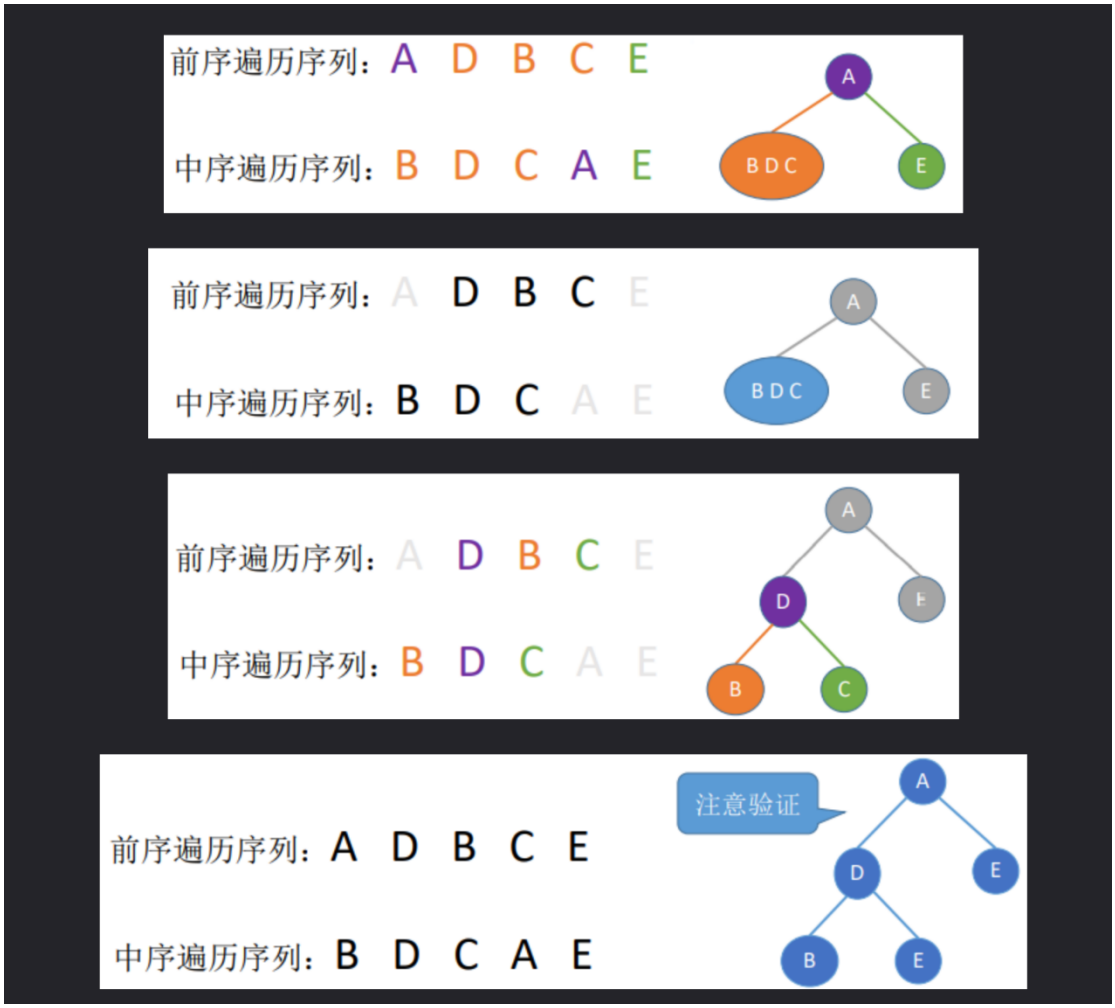
https://blog.csdn.net/shui2104

七、如何由遍历序列构造一颗二叉树?

（一） 前序和中序：

PS：前中后指的是根节点的位置。

通过前序找到根节点，找到该字母在中序的位置，左边是他的左子树右边是他的右子树，之后我们把左子树们带入前序，看谁在最前方谁就是根节点，以此类推。

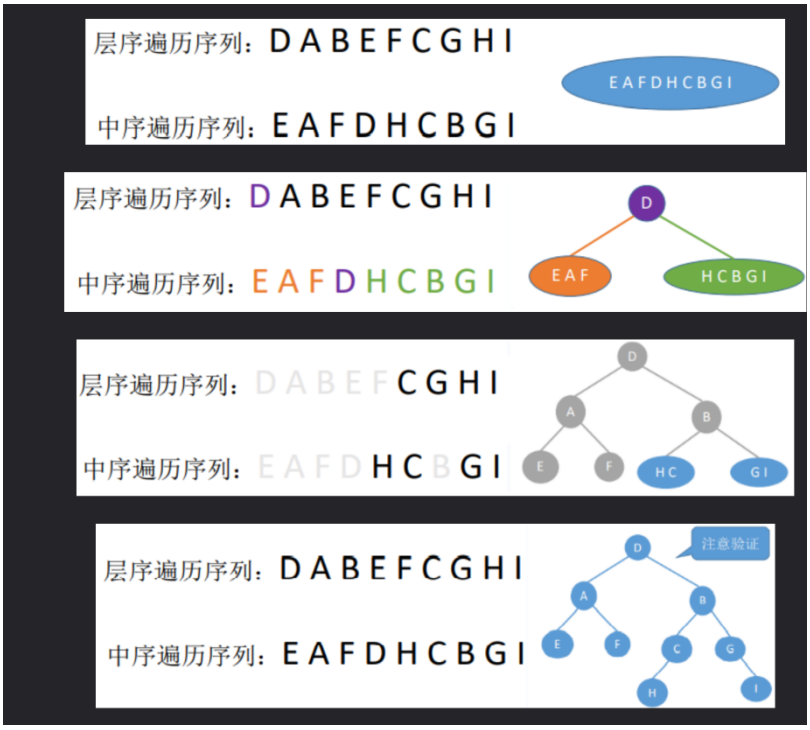


（二） 后序+中序：根节点的位置在最后了。

（三） 层序+中序：

PS：层序应该是 BFS。

层序因为是一层一层来的所以下面一层一定全在上面一层的下方出现，第一个字母是根节点，结合中序的左右进行判断。



八、哈希表的概念、构造方法、哈希有几种类型？

哈希表即散列存储结构，通过把关键码值映射到表中一个位置来访问记录。

构造好的哈希函数

- 所选函数尽可能简单，以便提高转换速度；
- 所选函数对关键码计算出的地址，应在哈希地址集中大致均匀分布，以减少空间浪费。

构造哈希函数的原则

1. 执行速度(即计算哈希函数所需的时间)
2. 关键字的长度
3. 哈希表的大小
4. 关键字的分布情况
5. 查找频率

构造方法：

- 直接定址法

Hash(key)=a * key+b (a、b 为常数)

优点：以关键码 key 的某个线性函数值为哈希地址，不会产生冲突。

缺点：要占用连续地址空间，空间效率低。

●除留余数法

Hash(key)=key mod p (p 是一个整数)

特点：以关键码除以 p 的余数作为哈希地址。

关键：如何选取合适的 p？

技巧：若设计的哈希表长为 m，则一般取 p≤m 且为质数(也可以是不包含小于 20 质因子的合数)

●数字分析法

特点：某关键字的某几位组为哈希地址。所选的位应当是：各种符号在该位上出现的频率大致相同。不太常用，除非非常有规律。

●平方取中法

特点：对关键码平方后，按哈希表大小，取中间的若干位作为哈希地址

理由：因为中间几位与数据的每一位都相关

例：

2589 的平方值为 6702921，可以取中间的 029 为地址

●折叠法

特点：将关键码自左到右分成位数相等的几部分（最后一部分位数可以短些），然后将这几部分叠加求和，并按哈希表表长，取后几位作为哈希地址。

适用于：每一位上各符号出现概率大致相同的情况

●随机数法

Hash(key)=random(key) (random 为随机函数)

适用于：关键字长度不等的情况，造表和查找都很方便

九、如何解决哈希冲突

1、开放地址法(再散列法)

开放定址法就是一旦发生了冲突，就去寻找下一个空的散列地址，只要散列表足够大，空的散列地址总能找到，并将记录存入

这种方法也称再散列法，其基本思想是：当关键字 key 的哈希地址 p=H（key）出现冲突时，以 p 为基础，产生另一个哈希地址 p1，如果 p1 仍然冲突，再以 p 为基础，产生另一个哈希地址 p2，…，直到找出一个不冲突的哈希地址 pi ，将相应元素存入其中。

就是说当发生冲突时，就去寻找下一个空的地址把数据存入其中，只要哈希表足够大，就总能找到，比如在设一个函数来控制这个奇偶分段。

2、**再哈希法** Rehash

再哈希法又叫双哈希法，有多个不同的 Hash 函数，当发生冲突时，使用第二个，第三个，….，等哈希函数计算地址，直到无冲突。虽然不易发生聚集，但是增加了计算时间。

3、**链地址法（HashMap 采用的这个方法）**

每个哈希表节点都有一个 next 指针，多个哈希表节点可以用 next 指针构成一个单向链表，被分配到同一个索引上的多个节点可以用这个单向链表连接起来，因而查找、插入和删除主要在同义词链中进行。链地址法适用于经常进行插入和删除的情况。

4、建立一个公共溢出区

在创建哈希表的同时，再额外创建一个公共溢出区，专门用来存放发生哈希冲突的元素。查找时，先从哈希表查，查不到再去公共溢出区查。

十、栈和队列的区别和内存结构

② 内存分配中的堆和栈

在 C 语言中，内存分配方式不外乎有如下三种形式：

1.从静态存储区域分配：它是由编译器自动分配和释放的，即内存在程序编译的时候就已经分配好，这块内存在程序的整个运行期间都存在，直到整个程序运行结束时才被释放，如全局变量与 static 变量。

2.在栈上分配：它同样也是由编译器自动分配和释放的，即在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元将被自动释放。需要注意的是，栈内存分配运算内置于处理器的指令集中，它的运行效率一般很高，但是分配的内存容量有限。

3.从堆上分配：也被称为动态内存分配，它是由程序员手动完成申请和释放的。即程序在运行的时候由程序员使用内存分配函数（如 malloc 函数）来申请任意多少的内存，使用完之后再由程序员自己负责使用内存释放函数（如 free 函数）来释放内存。也就是说，动态内存的整个生存期是由程序员自己决定的，使用非常灵活。需要注意的是，如果在堆上分配了内存空间，就必须及时释放它，否则将会导致运行的程序出现内存泄漏等错误。

JAVA

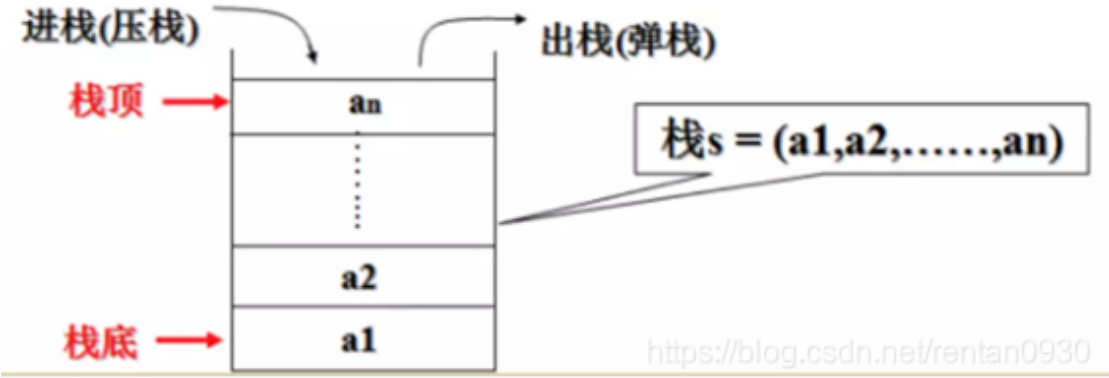
Java中内存主要包含4块，即heap(堆内存)、stack(栈内存)、data segment(静态变量或是常量存放区)、codesegment(方法区)

堆内存中存放的是new出的对象，new出的对象只包含成员变量。
栈内存中：存放的是局部成员变量。对于基本的数据类型存放的是基本变量的值，而对于对象变量，存放的是堆内存的地址。
静态、常量区：存放的是静态变量(类变量)或是常量。
方法区：存放的是对象的方法。因此即使new出多个对象也是只是存在一个方法。

如果创建一个对象，对象存放在堆中，而栈中存放了指向其的地址。如果是基本变量就直接存放在栈中。

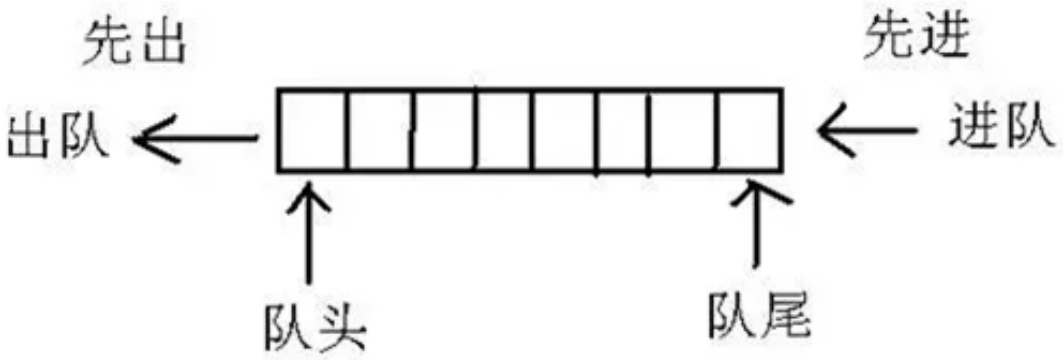
栈是限定仅在表尾进行插入和删除操作的线性表。允许插入和删除的一端称为栈顶，另一端称为栈底，插入和删除只能在栈顶进行。而且，栈是一种具有后进先出的数据结构，又称为后进先出的线性表，简称 LIFO（Last In First Out）结构。

栈的应用-递归



队列是只允许在一端进行插入操作、而在另一端进行删除操作的线性表。允许插入的一端称为队尾，允许删除的一端称为队头。和栈一样，队列是一种操作受限制的线性表。队列是一种先进先出的数据结构，又称为先进先出的线性表，简称 FIFO（First In First Out）结构。

队列应用-打印机等需要按序排队的。



堆是一种经过排序的树形数据结构，每个节点都有一个值，通常我们所说的堆的数据结构是指二叉树，分为大顶堆和小顶堆。

十一、数据结构中的栈和计算机里的栈有什么不同

1、数据结构中的“栈”是一个概念，是逻辑存在的，是管理数据的一种手段和方法。

2、计算机操作系统中的栈是指一块内存区域，该区域的管理(内存空间的分配与回收)采用类似数据结构中“栈”的特点进行操作。操作系统中的栈是物理存在的，是对数据结构中的栈这种手段的实现。

无论哪一种栈，都遵循“后进先出”的特点。

十二、如何区分循环队列是队空还是队满?

一般来说 rear==front 只会出现两种情况队满或者队空

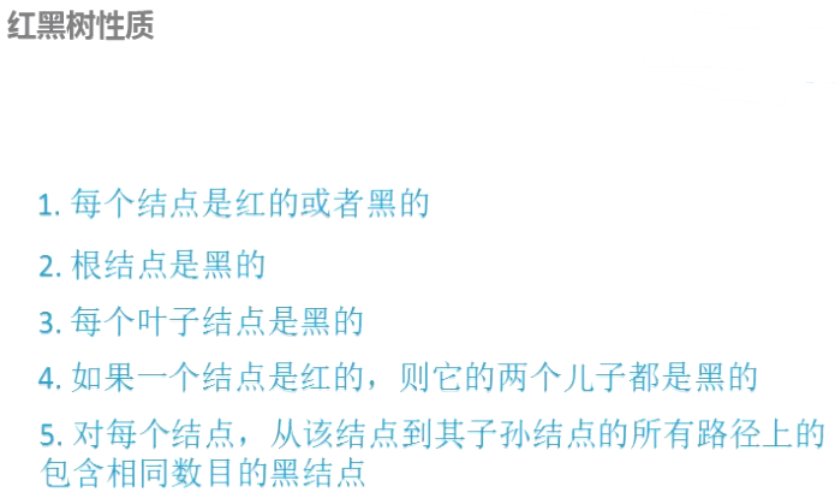
两种方式：

1、牺牲一个空间，那么空就是 rear==front，满就是（rear+1）%n==front。

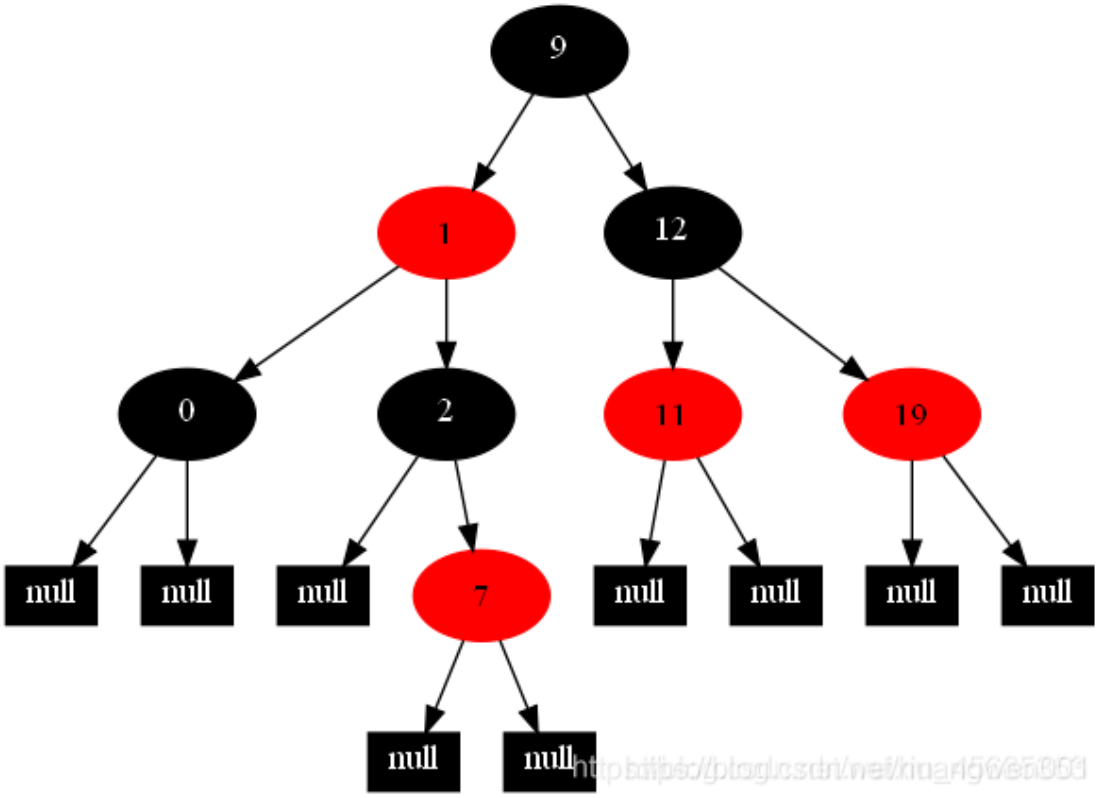
2、设置 tag 变量。Tag=-1，有数据出队，tag=1 时有数据入队。这样 rear==front 时 tag=-1，则为空，tag=1 时则为满。

十三、红黑树

红黑树是一种平衡二叉树的变体，它的左右子树高差有可能大于 1，所以红黑树不是严格意义上的平衡二叉树（AVL），其平均查找性能强于 AVL，他是自平衡二叉树。主要是用它来存储有序的数据，复杂度 O(lgn)



特性(5)，确保没有一条路径会比其他路径长出俩倍。因而，红黑树是相对是接近平衡的二叉树。



由于这些性质的约定，红黑树没有一条路径会比其他路径长出两倍，因而红黑树是近似平衡的。

十四、八大排序

三、选择排序

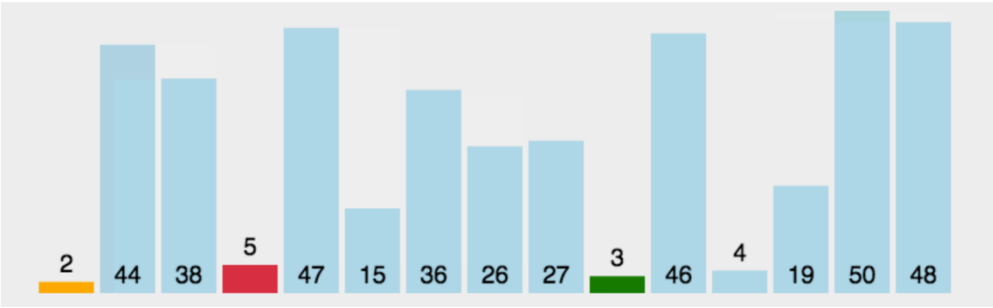
思路：

第一轮，找到最小的元素，和数组第一个数交换位置。

第二轮，找到第二小的元素，和数组第二个数交换位置...

直到最后一个元素，排序完成。

动画演示：

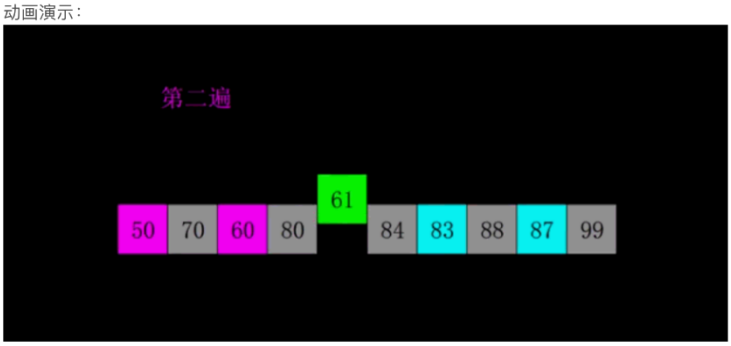


四、希尔排序

思路：

把数组分割成若干(h)个小组(一般数组长度length/2)，然后对每一个小组分别进行插入排序。每一轮分割的数组的个数逐步缩小，h/2->h/4->h/8，并且进行排序，保证有序。当h=1时，则数组排序完成。

动画演示：



实现代码：

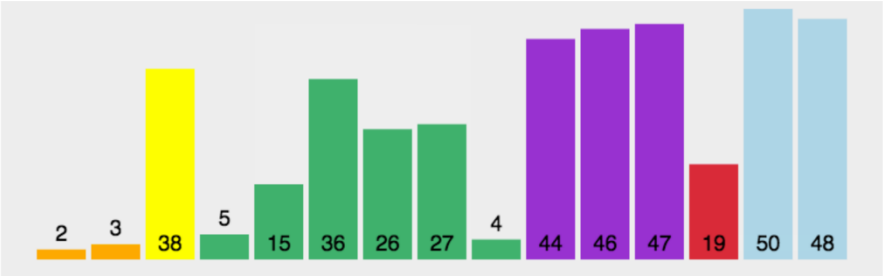
五、快速排序

快排，面试最喜欢问的 排序算法。这是运用分治法的一种排序算法。

思路：

1. 从数组中选一个数做为基准值，一般选第一个数，或者最后一个数。
2. 采用双指针(头尾两端)遍历，从左往右找到比基准值大的第一个数，从右往左找到比基准值小的第一个数，交换两数位置，直到头尾指针相等或头指针大于尾指针，把基准值与头指针的数交换。这样一轮之后，左边的数就比基准值小，右边的数就比基准值大。
3. 对左边的数列，重复上面1，2步骤。对右边重复1，2步骤。
4. 左右两边数列递归结束后，排序完成。

动画演示：



排序方法	时间复杂度			空间复杂度	稳定性
	平均复杂度	最好情况	最坏情况		
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定 java技术爱好者
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
希尔排序	$O(n^{1.3})$	$O(n)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(nlogn)$	$O(nlogn)$	$O(n^2)$	$O(1)$	不稳定
归并排序	$O(nlogn)$	$O(nlogn)$	$O(nlogn)$	$O(n)$	稳定
堆排序	$O(nlogn)$	$O(nlogn)$	$O(nlogn)$	$O(1)$	不稳定
桶排序	$O(M+N)$	$O(M+N)$	全部落在一个桶，取决于桶内的排序方法	$O(M+N)$	取决于桶内的排序方法

基数排序

按位比较，是非比较算法，时间复杂度为 $O(n)$，稳定。

十五、哈夫曼树

25.哈夫曼树和哈夫曼编码：

1．哈夫曼树的定义

在许多应用中，树中结点常常被赋予一个表示某种意义的数值，称为该结点的权。从树的根到任意结点的路径长度（经过的边数）与该结点上权值的乘积，称为该结点的带权路径长度。树中所有叶结点的带权路径长度之和称为该树的带权路径长度，记为

$$\mathrm{WPL}=\sum_{i=1}^{n} w_i l_i$$

式中，w_i是第 i 个叶结点所带的权值，l_i是该叶结点到根结点的路径长度。

在含有 n 个带权叶结点的二叉树中，其中带权路径长度（WPL）最小的二叉树称为哈夫曼树，也称最优二叉树。例如，图 5.34 中的 3 棵二叉树都有 4 个叶子结点 a, b, c, d，分别带权 7, 5, 2, 4，它们的带权路径长度分别为

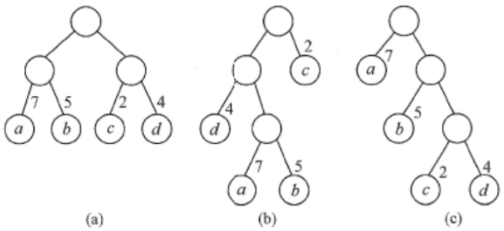


图 5.34　具有不同带权长度的二叉树

(a) WPL = 7×2 + 5×2 + 2×2 + 4×2 = 36。

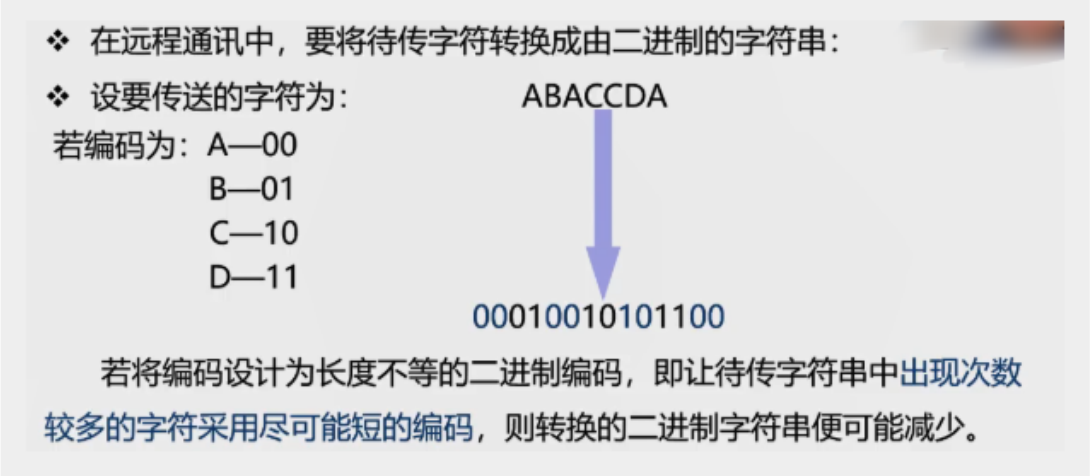
(b) WPL = 4×2 + 7×3 + 5×3 + 2×1 = 46。

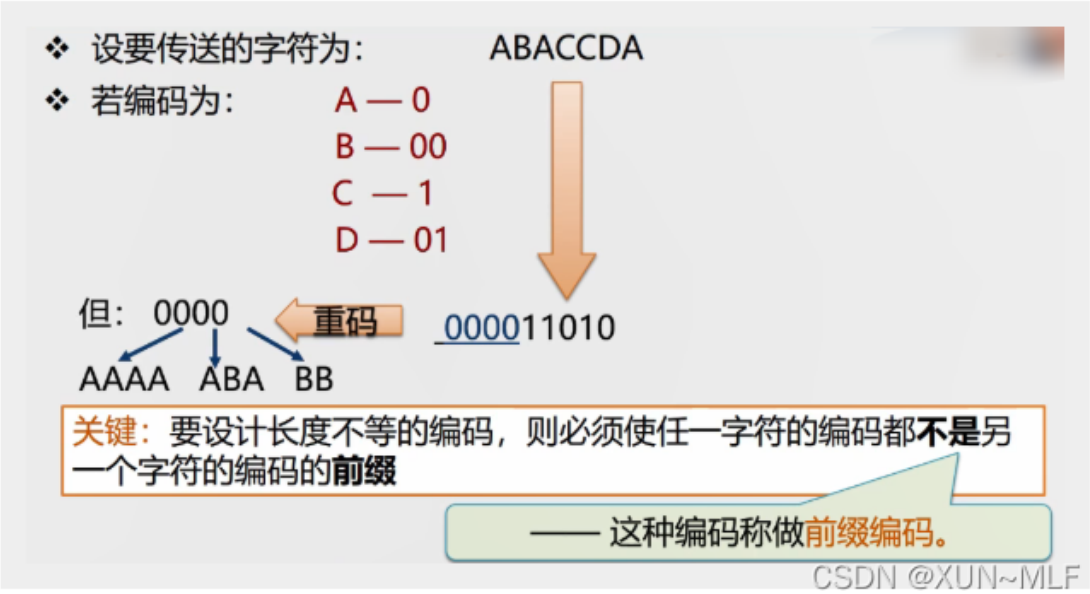
(c) WPL = 7×1 + 5×2 + 2×3 + 4×3 = 35。

其中，图 5.34(c)树的 WPL 最小。可以验证，它恰好为哈夫曼树。

每次选择两个最小权值的节点作为左右子树，他们的根节点权值为这两点相加，如 2+4=6

可以用于编码





十六、**链表反转**

1、使用栈结构来反转，时间和空间开销不立理想

2、使用三指针来反转，效率高

十七、贪心，动态规划，分治

贪心就利用局部的最优解来得到一个整体的结果，它不能保证整体最优。它的结果与选择的贪心策略有关

分治是将原问题分解为 n 个规模小，和原问题类似的子问题，通过子问题的解合并为原问题的解

动态规划也是一种分支，它可以利用前面的子问题的解来得到后面的子问题解，可以有效避免求重复的子问题。

在求解的过程中，保留可能得到的局部最优解，去掉其他局部解，解决到当

前的局部最优解。最终得到全局最优的解。

十八、如何用两个栈实现队列，如何用两个队列实现栈？

1、两个栈实现队列

假设初始两个栈为空；一个栈 A 用于维护 add 数据，另一个用于维护 poll 数据；当需要 add 的时候，直接添加到栈 A 的顶端即可；如果要 poll，直接从栈 B 弹出数据，但是如果栈 B 为空，则需要将 A 中所有的对象，依次弹出存入 B 中，然后再进行 poll

2、两个队列实现栈：假设初始两个队列为空，维护一个存入元素个数的变量 n；当需要 push 元素的时候，直接加入到非空的队列（都为空就任加一个）；当需要 pop 元素的时候，将非空的队列依次取出前 n-1 个元素，存入另一个空队列，然后将第 n 个元素取出即可，如此反复

十九、循环引用怎么解决

循环引用指两个对象相互强引用了对方，从而导致两个对象都无法被释放，引发了内存泄漏现象，互相引用变量的引用计数都为 1，本质上是引用计数的原因。

解决办法：只靠强引用计数方式，会存在循环引用的问题，导致对象永远无法被释放，弱引用就是专门用来解决循环引用问题的：

「若 A 强引用了 B，那 B 引用 A 时就需使用弱引用，当判断是否为无用对象时仅考虑强引用计数是否为 0，不关心弱引用计数的数量」

这样就解决了循环引用导致对象无法释放的问题，但这会引发野指针问题：当 B 要通过弱指针访问 A 时，A 可能已经被销毁了，那指向 A 的这个弱指针就变成野指针了。在这种情况下，就表示A确实已经不存在了，需要进行重新创建等其他操作

二十、什么是拓扑排序

在图论中，拓扑排序（TopologicalSorting）是一个有向无环图的所有顶点的线性序列。且该序列必须满足下面两个条件：每个顶点出现且只出现一次

若存在一条从顶点 A 到顶点 B 的路径，那么在序列中顶点 A 出现在顶点 B 的前面